Stratification: Monte Carlo and Simulation

Alban Géron,
Arnaud Barrat,
Camille Legrée-Hagbarth,
Tristan Fabre

April 29th, 2025

Introduction

We will study the integral of a periodic function which depends on the average of the input variables; more precisely, we have

$$f(u_1, \ldots, u_d) = \varphi\left(\frac{u_1 + \cdots + u_d}{d}\right) \qquad \text{where } \varphi : \mathbb{R} \to \mathbb{R} \text{ is periodic}$$

Such integrals exist in various applications:

- **Physics:** modeling collective behavior in oscillatory systems or wave-based phenomena.
- **Machine Learning:** using periodic kernels in Gaussian Processes or kernel methods to model cyclical patterns.
- **Finance:** modeling seasonality or cyclic risk premiums based on averaged economic indicators.
- **Numerical Analysis:** benchmark function for evaluating Monte Carlo and variance reduction methods.

Problem

- We seek to estimate the integral

$$I = \int_{[0,1]^d} f(u)du \qquad \text{where } f(u_1, \ldots, u_d) = \cos\left(2\pi\left(\frac{1}{d}\sum_{i=1}^{d} u_i - \frac{1}{2}\right)\right)$$

Problem

- We seek to estimate the integral

$$I = \int_{[0,1]^d} f(u)du \qquad \text{where } f(u_1, \ldots, u_d) = \cos\left(2\pi\left(\frac{1}{d}\sum_{i=1}^{d} u_i - \frac{1}{2}\right)\right)$$

- For $d = 1$: $I = \int_0^1 \cos\left(2\pi\left(u - \frac{1}{2}\right)\right) du = \frac{1}{2\pi}\int_{-\pi}^{\pi} \cos(t)dt = 0$.

Problem

- We seek to estimate the integral

$$I = \int_{[0,1]^d} f(u)du \qquad \text{where } f(u_1, \dots, u_d) = \cos\left(2\pi\left(\frac{1}{d}\sum_{i=1}^{d} u_i - \frac{1}{2}\right)\right)$$

- For $d = 1$: $I = \int_0^1 \cos\left(2\pi\left(u - \frac{1}{2}\right)\right) du = \frac{1}{2\pi}\int_{-\pi}^{\pi} \cos(t)dt = 0$.
- For large values of d: we have

$$\begin{aligned} I &= \int_{[0,1]^d} \varphi\left(\frac{u_1 + \cdots + u_d}{d}\right) du_1 \cdots du_d && \text{where } \varphi(t) = \cos\left(2\pi\left(t - \frac{1}{2}\right)\right) \\ &= \mathbb{E}\left[\varphi\left(\frac{U_1 + \cdots + U_d}{d}\right)\right] && \text{where } U_1, \dots, U_d \overset{\text{i.i.d.}}{\sim} \mathcal{U}([0,1]) \end{aligned}$$

which converges to $\varphi(\mathbb{E}[U_1]) = 1$ by the law of large numbers, the continuous mapping theorem (φ continuous) and the dominated convergence theorem (φ bounded).

Summary

1. Estimation methods used to solve the problem (Monte-Carlo, quasi-Monte-Carlo)

2. Implementation of unbiased estimators using N. CHOPIN's paper

3. Implementation of importance sampling methods

Monte-Carlo

- Notice that $I = \mathbb{E}[f(U)]$, where $U \sim \mathcal{U}([0,1]^d)$.

Monte-Carlo

- Notice that $I = \mathbb{E}[f(U)]$, where $U \sim \mathcal{U}([0,1]^d)$.
- Thus, following Monte-Carlo's method, we approximate such an expectation by

$$\frac{1}{N}\sum_{n=1}^{N} f(U_n)$$

where N is a large integer and the U_n's are N i.i.d. random vectors uniformly distributed on $[0,1]^d$.

Quasi-Monte-Carlo

- In the course's slides, quasi-Monte-Carlo has been proven to be more efficient in terms of computation than Monte-Carlo, under weak assumptions.

Quasi-Monte-Carlo

- In the course's slides, quasi-Monte-Carlo has been proven to be more efficient in terms of computation than Monte-Carlo, under weak assumptions.
- Again, we approximate $I = \mathbb{E}[f(U)]$ by an empirical mean of the form $\frac{1}{N}\sum_{n=1}^{N} f(U_n)$. However here the U_n's are generated in the set $\mathfrak{C}_N = \left\{\frac{1}{2N}, \frac{3}{2N}, \ldots, \frac{2N-1}{2N}\right\}^d$ under the condition (C):

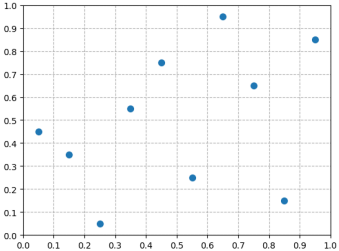
$$\text{each one of the points } U_1, \ldots, U_N \text{ falls on exactly one strip of the set } \mathfrak{C}_N \qquad \text{(C)}$$

Quasi-Monte-Carlo

- In the course's slides, quasi-Monte-Carlo has been proven to be more efficient in terms of computation than Monte-Carlo, under weak assumptions.
- Again, we approximate $I = \mathbb{E}[f(U)]$ by an empirical mean of the form $\frac{1}{N}\sum_{n=1}^{N} f(U_n)$. However here the U_n's are generated in the set $\mathfrak{C}_N = \left\{\frac{1}{2N}, \frac{3}{2N}, \ldots, \frac{2N-1}{2N}\right\}^d$ under the condition (C):

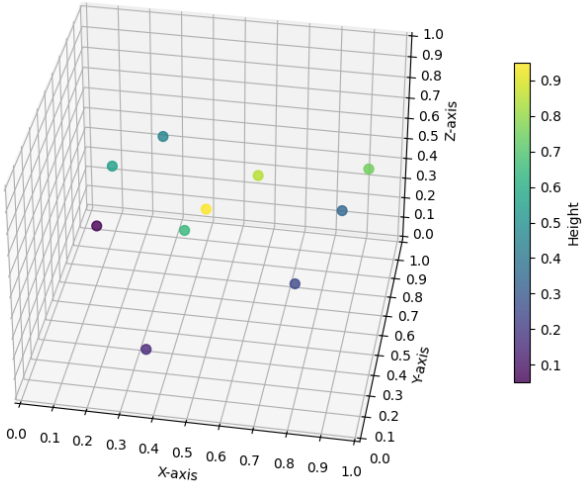
$$\text{each one of the points } U_1, \ldots, U_N \text{ falls on exactly one strip of the set } \mathfrak{C}_N \qquad \text{(C)}$$

- For $d = 2$: the condition (C) means that there is exactly one point in each line and each column:



Quasi-Monte-Carlo ($d = 3$)

- For $d = 3$:



Quasi-Monte-Carlo (formalization)

- (C) boils down to generating a d-dimensional table $M = (m_{i_1,\dots,i_d}) \in \mathbb{R}^{N^d}$ such that

$$m_{i_1,\dots,i_d} = \mathbb{1}_{i_1=\sigma_2(i_2)=\cdots=\sigma_d(i_d)}, \qquad \forall (i_1,\dots,i_d) \in \{0,1,\dots,N-1\}^d$$

  where $\sigma_2,\dots,\sigma_d$ are $d-1$ permutations of $\{0,1,\dots,N-1\}$.

Quasi-Monte-Carlo (formalization)

- (C) boils down to generating a d-dimensional table $M = (m_{i_1,\dots,i_d}) \in \mathbb{R}^{N^d}$ such that

$$m_{i_1,\dots,i_d} = \mathbb{1}_{i_1=\sigma_2(i_2)=\cdots=\sigma_d(i_d)}, \qquad \forall (i_1,\dots,i_d) \in \{0,1,\dots,N-1\}^d$$

  where $\sigma_2,\dots,\sigma_d$ are $d-1$ permutations of $\{0,1,\dots,N-1\}$.
- `numpy.random.permutation([0, 1, ..., N - 1])`: a permutation of $\{0,1,\dots,N-1\}$ (unidimensional array of size N) generated uniformly on the (finite) set $\mathfrak{S}(\{0,1,\dots,N-1\})$.

Quasi-Monte-Carlo (formalization)

- (C) boils down to generating a d-dimensional table $M = (m_{i_1,\dots,i_d}) \in \mathbb{R}^{N^d}$ such that

$$m_{i_1,\dots,i_d} = \mathbb{1}_{i_1=\sigma_2(i_2)=\cdots=\sigma_d(i_d)}, \qquad \forall (i_1,\dots,i_d) \in \{0,1,\dots,N-1\}^d$$

  where $\sigma_2,\dots,\sigma_d$ are $d-1$ permutations of $\{0,1,\dots,N-1\}$.
- `numpy.random.permutation([0, 1, ..., N - 1])`: a permutation of $\{0,1,\dots,N-1\}$ (unidimensional array of size N) generated uniformly on the (finite) set $\mathfrak{S}(\{0,1,\dots,N-1\})$.
- Then, we get the coordinates of the 'ones' as follows. For all $i_1 = 0,1,\dots,N-1$, we get **the only '1' that has coordinate i_1 on the first axis**, whose coordinates are

$$i_1, \sigma_2^{-1}(i_1), \dots, \sigma_d^{-1}(i_1)$$

Haber's Estimators: Intuition

- We want to approximate:

$$I(f) := \int_{[0,1]^d} f(u)\, du$$

where

$$f(u) = \cos\left(2\pi\left(\frac{1}{d}\sum_{i=1}^{d} u_i - \frac{1}{2}\right)\right)$$

Haber's Estimators: Intuition

- We want to approximate:

$$I(f) := \int_{[0,1]^d} f(u)\, du$$

where

$$f(u) = \cos\left(2\pi\left(\frac{1}{d}\sum_{i=1}^{d} u_i - \frac{1}{2}\right)\right)$$

- **Idea:** Use a stratified sampling scheme based on a regular grid and introduce a random perturbation :

$$\begin{aligned}\mathfrak{C}_k &:= \left\{\left(\frac{2j_1+1}{2k}, \ldots, \frac{2j_d+1}{2k}\right) \quad \text{s.t.} \quad (j_1, \ldots, j_d) \in \{0, 1, \ldots, k-1\}^d\right\} \\ &= \left\{\frac{1}{2k}, \frac{3}{2k}, \ldots, \frac{2k-1}{2k}\right\}^d\end{aligned}$$

Haber's Estimators: Intuition

- We want to approximate:

$$I(f) := \int_{[0,1]^d} f(u)\, du$$

where

$$f(u) = \cos\left(2\pi\left(\frac{1}{d}\sum_{i=1}^{d} u_i - \frac{1}{2}\right)\right)$$

- **Idea:** Use a stratified sampling scheme based on a regular grid and introduce a random perturbation :

$$\begin{aligned}\mathfrak{C}_k &:= \left\{\left(\frac{2j_1+1}{2k}, \ldots, \frac{2j_d+1}{2k}\right) \quad \text{s.t.} \quad (j_1, \ldots, j_d) \in \{0, 1, \ldots, k-1\}^d\right\} \\ &= \left\{\frac{1}{2k}, \frac{3}{2k}, \ldots, \frac{2k-1}{2k}\right\}^d\end{aligned}$$

- d is the dimension of the hypercube $[0,1]^d$ and k the level of stratification.

Haber's Estimators: Intuition

- We want to approximate:

$$I(f) := \int_{[0,1]^d} f(u)\, du$$

where

$$f(u) = \cos\left(2\pi\left(\frac{1}{d}\sum_{i=1}^{d} u_i - \frac{1}{2}\right)\right)$$

- **Idea:** Use a stratified sampling scheme based on a regular grid and introduce a random perturbation :

$$\begin{aligned}\mathfrak{C}_k &:= \left\{\left(\frac{2j_1+1}{2k}, \ldots, \frac{2j_d+1}{2k}\right) \quad \text{s.t.} \quad (j_1, \ldots, j_d) \in \{0, 1, \ldots, k-1\}^d\right\} \\ &= \left\{\frac{1}{2k}, \frac{3}{2k}, \ldots, \frac{2k-1}{2k}\right\}^d\end{aligned}$$

- d is the dimension of the hypercube $[0,1]^d$ and k the level of stratification.
- Grid of size k^d.

Haber's Estimators: Intuition

Each point $c \in \mathfrak{C}_k$ is the centre of a cube of side $\frac{1}{k}$.

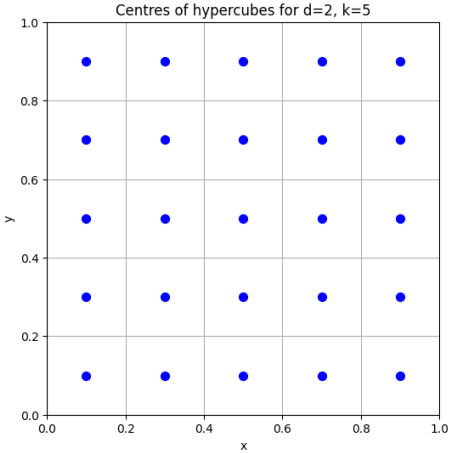


Figure: Principle of Haber's stratification

Haber vs. Quasi-Monte Carlo

- **Quasi-Monte Carlo (QMC):**
  - Uses low-discrepancy sequences to fill $[0,1]^d$ more uniformly than i.i.d. random points.
  - **Deterministic:** no randomness in sample generation.
  - **RMSE Convergence:** The Root Mean Squared Error (RMSE) convergences at rate $\mathcal{O}(N^{-1}(\log N)^{\frac{d}{2}})$ for functions of bounded variation.

Haber vs. Quasi-Monte Carlo

- **Quasi-Monte Carlo (QMC):**
  - Uses low-discrepancy sequences to fill $[0,1]^d$ more uniformly than i.i.d. random points.
  - **Deterministic:** no randomness in sample generation.
  - **RMSE Convergence:** The Root Mean Squared Error (RMSE) convergences at rate $\mathcal{O}(N^{-1}(\log N)^{\frac{d}{2}})$ for functions of bounded variation.

- **Haber Estimators:**
  - Use regular grid stratification $\mathfrak{C}_k \subset [0,1]^d$ with **random perturbations** in each cell.
  - **Unbiased Estimator:** Stratification minimizes bias by ensuring that each estimate is representative of the function's behavior over each subdomain.
  - **Optimal RMSE Convergence:** RMSE converges at rate $\mathcal{O}(N^{-1/2-r/d})$.
  - **Minimization of the Variance:** subdomain are chosen to be as homogeneous as possible, thereby reducing the fluctuations of the function $f(u)$ within each subdomain.

Haber's estimator of order 1

- Haber's estimator of order 1 is defined by

$$\hat{I}_{1,k}(f) := \frac{1}{k^d} \sum_{c \in \mathfrak{C}_k} f(c + U_c), \quad U_c \sim \mathcal{U}\left(\left[-\frac{1}{2k}, \frac{1}{2k}\right]^d\right)$$

Haber's estimator of order 1

- Haber's estimator of order 1 is defined by

$$\hat{I}_{1,k}(f) := \frac{1}{k^d} \sum_{c \in \mathfrak{C}_k} f(c + U_c), \quad U_c \sim \mathcal{U}\left(\left[-\frac{1}{2k}, \frac{1}{2k}\right]^d\right)$$

- Instead of randomly dropping samples everywhere, this method **ensures a uniform spatial coverage** by stratifying the domain into small subcubes.

Haber's estimator of order 1

- Haber's estimator of order 1 is defined by

$$\hat{I}_{1,k}(f) := \frac{1}{k^d} \sum_{c \in \mathfrak{C}_k} f(c + U_c), \quad U_c \sim \mathcal{U}\left(\left[-\frac{1}{2k}, \frac{1}{2k}\right]^d\right)$$

- Instead of randomly dropping samples everywhere, this method **ensures a uniform spatial coverage** by stratifying the domain into small subcubes.
- The perturbation U_c inside each hypercube introduces **randomness**, allowing us to sample the function f without relying solely on the midpoints.

Haber's estimator of order 1

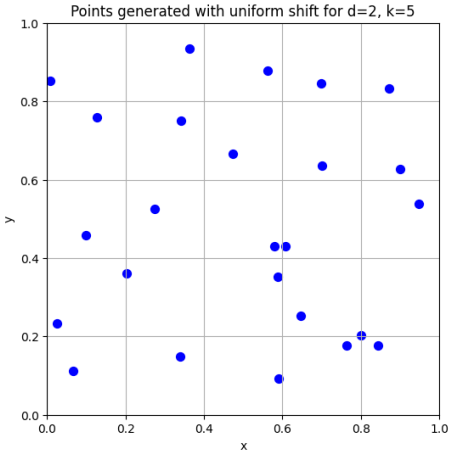


Figure: Stratification of Haber's estimator of order 1

Haber's estimator of order 2

- Let

$$g_c(u) := \frac{f(c+u) + f(c-u)}{2}$$

Haber's estimator of order 2 is defined by

$$\hat{I}_{2,k}(f) := \frac{1}{k^d} \sum_{c \in \mathfrak{C}_k} g_c(U_c), \quad U_c \sim \mathcal{U}\left(\left[-\frac{1}{2k}, \frac{1}{2k}\right]^d\right)$$

Haber's estimator of order 2

- Let
$$g_c(u) := \frac{f(c+u) + f(c-u)}{2}$$
Haber's estimator of order 2 is defined by
$$\hat{I}_{2,k}(f) := \frac{1}{k^d} \sum_{c \in \mathfrak{C}_k} g_c(U_c), \quad U_c \sim \mathcal{U}\left(\left[-\frac{1}{2k}, \frac{1}{2k}\right]^d\right)$$
- This estimator improves upon the first order by introducing a second level of perturbation within each subcube. This helps **increase the randomness** of the sampling process.

Haber's estimator of order 2

- Let
$$g_c(u) := \frac{f(c+u) + f(c-u)}{2}$$
Haber's estimator of order 2 is defined by
$$\hat{I}_{2,k}(f) := \frac{1}{k^d} \sum_{c \in \mathfrak{C}_k} g_c(U_c), \quad U_c \sim \mathcal{U}\left(\left[-\frac{1}{2k}, \frac{1}{2k}\right]^d\right)$$
- This estimator improves upon the first order by introducing a second level of perturbation within each subcube. This helps **increase the randomness** of the sampling process.
- The second perturbation introduces additional variability in the sampling points, making the estimator more robust and **reducing bias**.

Haber's estimator of order 2

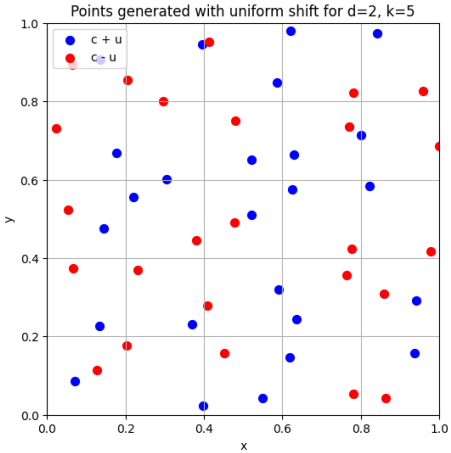


Figure: Stratification of Haber's estimator of order 2

Comparison of estimators

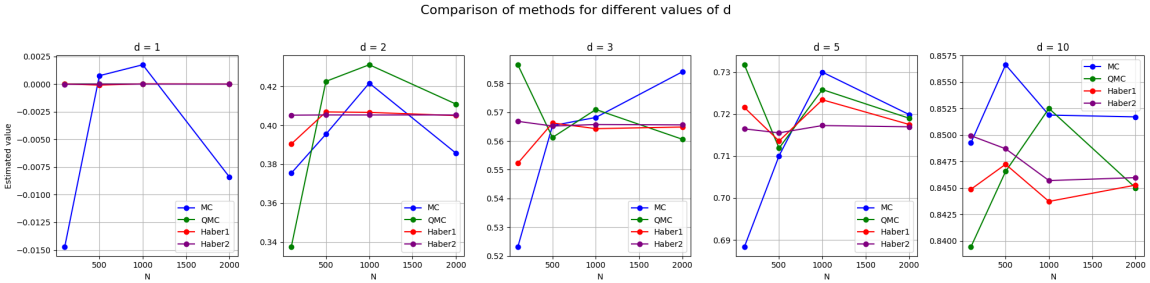


Figure: Comparison of estimators for different values of d and N

- To ensure fair comparison, we make the number of function evaluations comparable across methods by setting $k = N^{1/d}$ in the Haber estimators.

Limits of Haber Estimators

- **Curse of dimensionality:** convergence rate $\mathcal{O}(N^{-1/2-r/d})$ degrades with dimension d; slow for high-dimensional problems.

Limits of Haber Estimators

- **Curse of dimensionality:** convergence rate $\mathcal{O}(N^{-1/2-r/d})$ degrades with dimension d; slow for high-dimensional problems.
- **Regularity assumptions:** optimal rates assume $f \in \mathcal{C}^r([0,1]^d)$; not suitable for discontinuous or highly oscillatory functions.

Limits of Haber Estimators

- **Curse of dimensionality:** convergence rate $\mathcal{O}(N^{-1/2-r/d})$ degrades with dimension d; slow for high-dimensional problems.
- **Regularity assumptions:** optimal rates assume $f \in \mathcal{C}^r([0,1]^d)$; not suitable for discontinuous or highly oscillatory functions.
- **No adaptive refinement:** fixed stratification cannot focus sampling effort on regions where f varies more.

How does Importance Sampling work ?

- We want to approximate the following integral:

$$I = \int_{[0,1]^d} f(x)\, dx$$

How does Importance Sampling work ?

- We want to approximate the following integral:

$$I = \int_{[0,1]^d} f(x)\, dx$$

- We rewrite the integral as:

$$I = \int_{[0,1]^d} f(x)\, dx = \int_{[0,1]^d} \frac{f(x)}{g(x)} g(x)\, dx = \mathbb{E}_{X \sim g}\left[\frac{f(X)}{g(X)} \cdot \mathbb{1}_{X \in [0,1]^d}\right]$$

for any probability density function $g : \mathbb{R}^d \to \mathbb{R}$ satisfying $[0,1]^d \subseteq \mathrm{Supp}(g)$.[1] Such a density is called a **proposal density**.

[1] This assumption avoids division by zero.

How does Importance Sampling work ?

- We want to approximate the following integral:

$$I = \int_{[0,1]^d} f(x)\, dx$$

- We rewrite the integral as:

$$I = \int_{[0,1]^d} f(x)\, dx = \int_{[0,1]^d} \frac{f(x)}{g(x)} g(x)\, dx = \mathbb{E}_{X\sim g}\left[\frac{f(X)}{g(X)} \cdot \mathbb{1}_{X\in[0,1]^d}\right]$$

  for any probability density function $g : \mathbb{R}^d \to \mathbb{R}$ satisfying $[0,1]^d \subseteq \mathrm{Supp}(g)$.[1] Such a density is called a **proposal density**.

- Thus, instead of approximating $\mathbb{E}[f(U)]$ where $U \sim \mathcal{U}([0,1]^d)$, we now seek to approximate $\mathbb{E}\left[\frac{f(X)}{g(X)} \cdot \mathbb{1}_{X\in[0,1]^d}\right]$ where the random vector X is sampled from an appropriately chosen density g.

[1] This assumption avoids division by zero.

How does Importance Sampling work ?

- We then draw $x_1, x_2, \ldots, x_N \overset{\text{i.i.d.}}{\sim} g$ and estimate:

$$\hat{I} = \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i)}{g(x_i)} \cdot \mathbb{1}_{x_i \in [0,1]^d}$$

How does Importance Sampling work ?

- We then draw $x_1, x_2, \ldots, x_N \overset{\text{i.i.d.}}{\sim} g$ and estimate:

$$\hat{I} = \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i)}{g(x_i)} \cdot \mathbb{1}_{x_i \in [0,1]^d}$$

- The weight of each sample x_i is:

$$w_i = \frac{f(x_i)}{g(x_i)} \quad (\text{only if } x_i \in [0,1]^d)$$

Approximation of the integral using importance sampling

- Assume that the input vector $u = (u_1, \ldots, u_d)$ consists of independent components, each uniformly distributed over $[0, 1]$:

$$u_i \sim \mathcal{U}([0, 1]), \quad \text{for } i = 1, \ldots, d$$

Approximation of the integral using importance sampling

- Assume that the input vector $u = (u_1, \ldots, u_d)$ consists of independent components, each uniformly distributed over $[0, 1]$:

$$u_i \sim \mathcal{U}([0, 1]), \quad \text{for } i = 1, \ldots, d$$

- Then, for $i = 1, \ldots, d$, we have:

$$\mathbb{E}[u_i] = \frac{1}{2}, \quad \mathrm{Var}(u_i) = \frac{1}{12}$$

Approximation of the integral using importance sampling

- Assume that the input vector $u = (u_1, \ldots, u_d)$ consists of independent components, each uniformly distributed over $[0, 1]$:

$$u_i \sim \mathcal{U}([0,1]), \quad \text{for } i = 1, \ldots, d$$

- Then, for $i = 1, \ldots, d$, we have:

$$\mathbb{E}[u_i] = \frac{1}{2}, \quad \mathrm{Var}(u_i) = \frac{1}{12}$$

- By the Central Limit Theorem:

$$\sqrt{d}\left(\bar{u}_d - \frac{1}{2}\right) \xrightarrow[d\to\infty]{\mathcal{L}} \mathcal{N}\left(0, \frac{1}{12}\right) \quad \text{with } \bar{u}_d = \frac{1}{d}\sum_{i=1}^{d} u_i$$

Approximation of the integral using importance sampling

- Assume that the input vector $u = (u_1, \ldots, u_d)$ consists of independent components, each uniformly distributed over $[0, 1]$:

$$u_i \sim \mathcal{U}([0,1]), \quad \text{for } i = 1, \ldots, d$$

- Then, for $i = 1, \ldots, d$, we have:

$$\mathbb{E}[u_i] = \frac{1}{2}, \quad \mathrm{Var}(u_i) = \frac{1}{12}$$

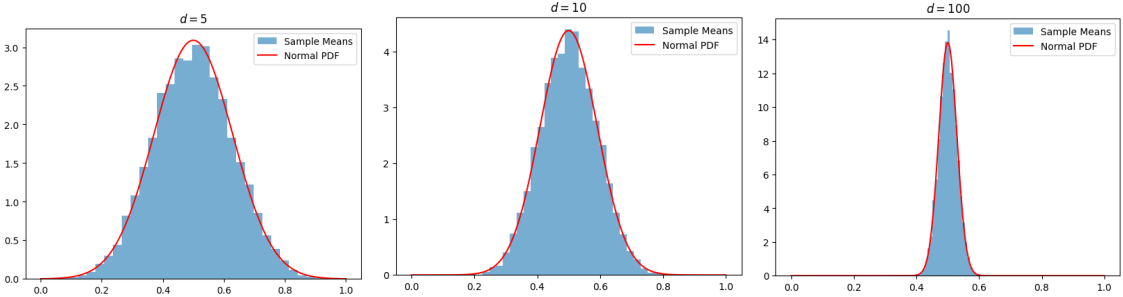
- By the Central Limit Theorem:

$$\sqrt{d}\left(\bar{u}_d - \frac{1}{2}\right) \xrightarrow[d\to\infty]{\mathcal{L}} \mathcal{N}\left(0, \frac{1}{12}\right) \quad \text{with } \bar{u}_d = \frac{1}{d}\sum_{i=1}^{d} u_i$$

- Consequently: for large values of d, the distribution of $\bar{u}_d$ is approximately $\mathcal{N}\left(\frac{1}{2}, \frac{1}{12d}\right)$.

Histogram of simulations with normal approximation

Overlaying the histogram of simulations of $\bar{u}_d$ from $n = 10,000$ simulations of $u_1, \ldots, u_d$, with the PDF of $\mathcal{N}\left(\frac{1}{2}, \frac{1}{12d}\right)$, shows that this is a great approximation.



Issue with the proposal density $g(x)$ in high dimensions

- The previous observation encourages us to consider a density such that the marginal law of each component is $\mathcal{N}(\mu, \sigma^2)$, where $\mu = \frac{1}{2}$ and $\sigma^2 = \frac{1}{12d}$.
  For instance: choose $g =$ the density of $\mathcal{N}_d\left(\left(\frac{1}{2}, \ldots, \frac{1}{2}\right), \frac{1}{12d} I_d\right)$.

Issue with the proposal density $g(x)$ in high dimensions

- The previous observation encourages us to consider a density such that the marginal law of each component is $\mathcal{N}(\mu, \sigma^2)$, where $\mu = \frac{1}{2}$ and $\sigma^2 = \frac{1}{12d}$.
  For instance: choose $g =$ the density of $\mathcal{N}_d\left(\left(\frac{1}{2}, \ldots, \frac{1}{2}\right), \frac{1}{12d} I_d\right)$.
- **Issue:**
  - When the dimension d is large, the proposal distribution $g(x)$ becomes extremely concentrated around its mean.

Issue with the proposal density $g(x)$ in high dimensions

- The previous observation encourages us to consider a density such that the marginal law of each component is $\mathcal{N}(\mu, \sigma^2)$, where $\mu = \frac{1}{2}$ and $\sigma^2 = \frac{1}{12d}$.
  For instance: choose $g =$ the density of $\mathcal{N}_d\left(\left(\frac{1}{2}, \ldots, \frac{1}{2}\right), \frac{1}{12d} I_d\right)$.
- **Issue:**
  - When the dimension d is large, the proposal distribution $g(x)$ becomes extremely concentrated around its mean.
  - As a result, the density values $g(x)$ become very large in a small region.

Issue with the proposal density $g(x)$ in high dimensions

- The previous observation encourages us to consider a density such that the marginal law of each component is $\mathcal{N}(\mu, \sigma^2)$, where $\mu = \frac{1}{2}$ and $\sigma^2 = \frac{1}{12d}$.
  For instance: choose $g =$ the density of $\mathcal{N}_d\left(\left(\frac{1}{2}, \ldots, \frac{1}{2}\right), \frac{1}{12d} I_d\right)$.
- **Issue:**
  - When the dimension d is large, the proposal distribution $g(x)$ becomes extremely concentrated around its mean.
  - As a result, the density values $g(x)$ become very large in a small region.
  - This leads to importance weights $w(x) = \frac{f(x)}{g(x)}$ that are extremely small, often close to zero, making the estimator numerically unstable or biased toward zero.

Why is $g(x_i)$ huge in high dimension? 1

- We are using a multivariate normal distribution:
  - centered at $\mu = 0.5$
  - variance $\sigma^2 = \frac{1}{12d}$

Why is $g(x_i)$ huge in high dimension? 1

- We are using a multivariate normal distribution:
  - centered at $\mu = 0.5$
  - variance $\sigma^2 = \frac{1}{12d}$
- So the density is:

$$g(x) = \prod_{j=1}^{d} \text{NormalPDF}(x_j; 0.5, \sigma)$$

Why is $g(x_i)$ huge in high dimension? 1

- We are using a multivariate normal distribution:
  - centered at $\mu = 0.5$
  - variance $\sigma^2 = \frac{1}{12d}$
- So the density is:

$$g(x) = \prod_{j=1}^{d} \text{NormalPDF}(x_j; 0.5, \sigma)$$

- Each `NormalPDF` looks like:

$$\frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{-\frac{(x_j - 0.5)^2}{2\sigma^2}}$$

Why is $g(x_i)$ huge in high dimension? 2

- $$g(x) = \prod_{j=1}^{d} \text{NormalPDF}(x_j; 0.5, \sigma) \Rightarrow g(x) \text{ becomes very large near } x = 0.5$$

Why is $g(x_i)$ huge in high dimension? 2

- $$g(x) = \prod_{j=1}^{d} \text{NormalPDF}(x_j; 0.5, \sigma) \Rightarrow g(x) \text{ becomes very large near } x = 0.5$$
- The smaller σ is:
  - the sharper the peak of the density
  - $g(x)$ becomes very large around 0.5

Why is $g(x_i)$ huge in high dimension? 2

- $$g(x) = \prod_{j=1}^{d} \text{NormalPDF}(x_j; 0.5, \sigma) \Rightarrow g(x) \text{ becomes very large near } x = 0.5$$
- The smaller σ is:
  - the sharper the peak of the density
  - $g(x)$ becomes very large around 0.5
- So in the core of the density:

$$g(x) \approx 10^{40},\ 10^{60}, \ldots \quad \Rightarrow \quad \frac{1}{g(x)} \approx 10^{-40},\ 10^{-60}$$

Consequence

- Even if:
  - $h(x) = \mathbb{1}_{X \in [0,1]^d}$
  - $x \in [0,1]^d$

  The weight:

$$w(x) = \frac{h(x)}{g(x)} \approx \frac{1}{g(x)} \ll 1$$

Consequence

- Even if:
  - $h(x) = \mathbb{1}_{X \in [0,1]^d}$
  - $x \in [0,1]^d$

  The weight:

$$w(x) = \frac{h(x)}{g(x)} \approx \frac{1}{g(x)} \ll 1$$

- $\Rightarrow$ Infinitesimal weight $\Rightarrow$ estimator becomes numerically close to 0

Consequence

- Even if:
  - $h(x) = \mathbb{1}_{X \in [0,1]^d}$
  - $x \in [0,1]^d$

  The weight:

$$w(x) = \frac{h(x)}{g(x)} \approx \frac{1}{g(x)} \ll 1$$

- $\Rightarrow$ Infinitesimal weight $\Rightarrow$ estimator becomes numerically close to 0

- Because f **only depends on the mean of the input**, and we're drawing values from a distribution that accurately reflects how that mean behaves, we don't need to adjust with weights — just averaging the function values gives a reliable estimate.

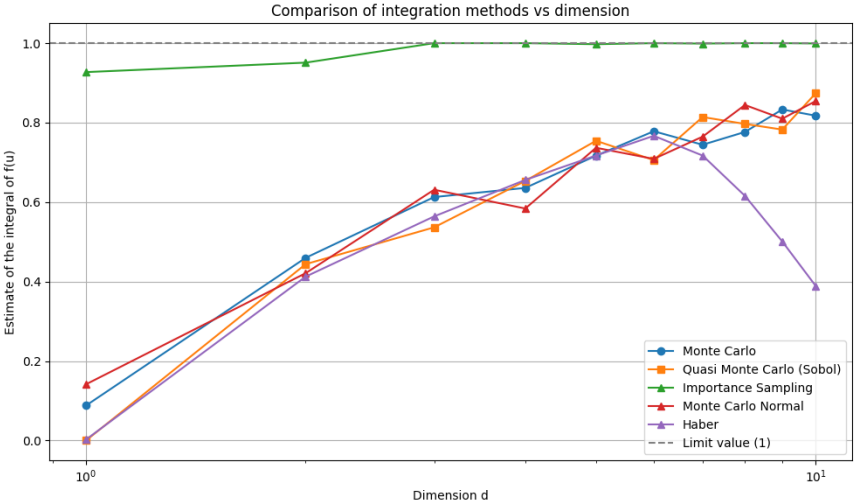
Testing the algorithm for multiple dimensions

We test importance sampling for different values of d, and visualize the distribution for $d = 10,000$ with weight fiex at 1.

In [22]:

```
for d in [2, 3, 4, 5, 6, 10, 20, 50, 100]:
    I = importance_sampling(f, d, 100)
    print(f"d = {d:>2} → Estimation ≈ {I:.6f}")
```

```
d =  2 → Estimation ≈ 0.959996
d =  3 → Estimation ≈ 0.978064
d =  4 → Estimation ≈ 0.999018
d =  5 → Estimation ≈ 0.998636
d =  6 → Estimation ≈ 0.999999
d = 10 → Estimation ≈ 0.999726
d = 20 → Estimation ≈ 0.999980
d = 50 → Estimation ≈ 0.999997
d = 100 → Estimation ≈ 1.000000
```

Comparison of integration methods vs dimension

Let's visualize the convergence on a graph: $N = 1024$,